Mauricio Salinas

Desarrollador Backend · Python · FastAPI · Django/DRF · Event-driven · Tooling para agentes de IA

Santiago, Chile · maundrex@gmail.com · +56 9 4758 6414
mauridev94.github.io · github.com/MauriDev94 · linkedin.com/in/mauricio-salinas-rubio
Disponible para trabajo remoto · Inglés C2

PERFIL

Desarrollador backend Python. Construyo APIs REST y sistemas event-driven con **FastAPI** y **Django/DRF** usando Clean Architecture, DDD y TDD: más de **700 tests** en mis tres proyectos principales, con integración contra PostgreSQL real en CI y decisiones documentadas en ADRs. Cofundador de **Publicala**, donde desarrollo una plataforma de pedidos con Next.js y Supabase. Construí un **harness** que no deja a los agentes de IA dar una tarea por terminada sin tests en verde.

EXPERIENCIA

Cofundador · Desarrollador Full-stack · Publicala — Santiago, Chile · oct 2026 – presente

Emprendimiento en etapa inicial: estudio web, de marketing y marca para pymes en Chile. Su primer producto está en desarrollo y aún no se lanza.

- Desarrollo una **plataforma multi-tenant de pedidos online** para restaurantes (tienda y panel del local en tiempo real) con **Next.js 16 y Supabase** (Postgres + RLS, Realtime) y arquitectura hexagonal por rebanadas verticales.
- Monté el **CI con GitHub Actions** (lint, tipos y tests en cada push) y el flujo de ramas y pull requests del equipo; implementé reglas de dominio del checkout con tests en Vitest.
- Especifico cada funcionalidad antes de programarla, con criterios de "listo": rechazo automático de pedidos (pg_cron), protección contra pedidos falsos y acceso de superadmin.

Analista de Soporte TI · Dominó (Cadaques) — Santiago, Chile · abr 2023 – jul 2024

- Construí un **ETL en Python (pandas) sobre SQL Server** que consolidó las **boletas emitidas en 2022–2023** por las empresas del holding: millones de documentos de un gran número de sucursales, recolectados, transformados y cargados de forma automatizada.
- Analicé y resolví incidencias en producción hasta su causa raíz, y escalé defectos al equipo de desarrollo con seguimiento hasta su resolución.
- Mantuve bases de datos SQL en producción, generé reportes para operaciones y automaticé tareas repetitivas con scripts Python.

PROYECTOS

Desarrollo independiente · abr 2025 – presente

Event-Driven Orders · Microservicios con RabbitMQ

> `3 microservicios · 138 tests + e2e · cobertura 85–92% por servicio · 19 ADRs`

Stack: Python 3.12 · FastAPI · RabbitMQ (aio-pika) · PostgreSQL 16 · Docker Compose · testcontainers · structlog · GitHub Actions

Órdenes e-commerce con tres servicios (órdenes, inventario y notificaciones) que se comunican solo por eventos.

- **Consumers idempotentes**: un evento reentregado no duplica efectos (`INSERT … ON CONFLICT DO NOTHING`).
- **Reserva de stock sin race conditions** (UPDATE condicional + bloqueo por fila), probada contra PostgreSQL real.
- **DLQ y reintentos con backoff** en 3 etapas (fallos transitorios vs. permanentes); consistencia eventual por coreografía.
- **Observabilidad**: logs JSON con correlation ID de punta a punta (HTTP → eventos → 3 servicios).

Repo: **github.com/MauriDev94/event-driven-orders**

MedBook API · Reservas médicas con Django + DRF

> `305 tests · 99.4% de cobertura con ramas · TDD`

Stack: Python 3.12 · Django 5.2 LTS · DRF 3.18 · PostgreSQL 16 · simplejwt · drf-spectacular · pytest · factory-boy · Docker · GitHub Actions

Los médicos publican horarios, el sistema genera slots y los pacientes reservan; cada cita sigue una máquina de estados (pending → confirmed → completed / cancelled / no-show).

- **RBAC** con 8 clases de permisos y control de acceso a nivel de objeto (médico / paciente / admin).

- **Reservas concurrentes seguras** (transacción + UPDATE atómico), con un test de dos hilos que compiten por el slot.
- ORM sin N+1 (`select_related` / `prefetch_related`), OpenAPI 3 con Swagger y rate limiting en el login.
- **TDD** (Red-Green-Refactor); CI con lint, tests contra PostgreSQL real y gate de cobertura del 95%.

Repo: **github.com/MauriDev94/MedBook** · Swagger: **medbook-oiio.onrender.com/api/docs** (primera carga ~50 s)

Monolith API · Monolito modular con FastAPI

> `263 tests · gate de cobertura con ramas en 85% · desplegada en Render`

Stack: Python 3.12 · FastAPI · SQLAlchemy 2.0 · Alembic · PostgreSQL 16 · PyJWT · Argon2 · Docker · GitHub Actions

API REST con Clean Architecture y DDD en tres módulos (auth, usuarios, tareas) y frontend propio en React + TS.

- **Auth completa**: JWT con rotación de refresh tokens y detección de reuso, Google SSO con protección CSRF (cookie `state` firmada), OTP como HMAC-SHA256, Argon2 y rate limiting por IP.
- Dominio sin dependencias de frameworks; módulos comunicados por puertos (`UserProvider`).
- Tests contra PostgreSQL real con las migraciones de Alembic, y un gate `alembic check` en CI que detecta divergencias entre modelo y migraciones.

Repo: **github.com/MauriDev94/Api_monolith** · Swagger: **api-monolith.onrender.com/docs** (primera carga ~50 s) ·
Frontend: **monolith-frontend-delta.vercel.app**

MauriDev Harness · Verificación determinista para agentes de código

Repo privado

> `Claude Code + OpenCode · 3 stacks · probado en 3 proyectos`

Stack: Python (solo librería estándar) · TypeScript · hooks de Claude Code · plugin de OpenCode · ruff · mypy · pytest

Un agente que dice "terminé, los tests pasan" afirma, no demuestra: este harness exige un exit code que lo respalde.

- **Núcleo determinista**: detecta el stack y corre ruff, mypy, pytest y cobertura en orden; el exit code es el veredicto.
- **Adaptadores delgados** para Claude Code (hooks `PostToolUse` y `Stop`) y OpenCode (plugin en TypeScript) que bloquean las ediciones que rompen el gate y el "listo" sin evidencia.
- Scaffolding de proyectos Python, FastAPI y Django que nacen en verde e instalador idempotente.

Otros proyectos

- **Tag_Bot**: descarga automática de boletas de autopistas desde Gmail (API de Gmail + scraping con Playwright) y envío por Telegram. Repo privado.

HABILIDADES TÉCNICAS

Lenguajes	Python, SQL, TypeScript
Backend	FastAPI, Django, Django REST Framework, Pydantic v2, SQLAlchemy 2.0, Alembic
Datos	PostgreSQL, SQL Server, pandas, RabbitMQ (aio-pika), Supabase
Arquitectura	Clean Architecture, DDD, monolito modular, microservicios event-driven, ADRs
Testing	pytest, TDD, testcontainers, factory-boy, cobertura con ramas, Vitest
Seguridad	JWT (rotación y detección de reuso), OAuth2 / Google SSO, OTP, Argon2, RBAC, rate limiting
DevOps	Docker, Docker Compose, GitHub Actions, Render, Vercel, structlog (logs JSON), correlation IDs
IA aplicada	Claude Code (skills, subagentes, hooks), OpenCode, verificación determinista para agentes
Prácticas	SOLID, Spec-Driven Development, Conventional Commits, pull requests y code review

IDIOMAS

Español: Nativo
Inglés: C2 (EF SET 71/100)

EDUCACIÓN

Analista Programador · Titulado
Instituto Profesional Santo Tomás · Santiago, Chile · mar 2021 – jul 2025